SAP AMS Transition App — Frontend Build Context (Updated with RBAC)

> Give this entire document to a new AI chat to start building the frontend

---

Who I Am & What I'm Doing

I am an intern building a web application called the **SAP AMS Knowledge Management & Expert Assistance Platform** for an internship project at NTT DATA Business Solutions. There are 2 interns total. My teammate has already built the backend. My job is to build the **React frontend** that connects to the backend APIs.

I am comfortable with React and Node.js but not very experienced. Please explain things clearly and give me complete code files, one at a time, so I can paste them in and test before moving to the next one.

What the App Does

This is an internal web tool used when a company switches its SAP IT support from one vendor to another. The core feature is a **Knowledge Base** — where SAP support articles are stored, searched, and served to users based on their role. It also has an AI chatbot (future phase) that answers questions using those articles.

The app has **Role-Based Access Control (RBAC)** — different users see different articles based on their role and department.

User Roles & What They Can Do

Role	What They Can Access
Employee	Published articles only, general articles
L1 Support	General/published articles
L2 Support	Critical articles
Expert	Articles in their expertise area, including drafts
Team Lead	Can review and approve/reject articles

Role	What They Can Access
Admin	Full access to everything

What Each Role Can DO in the UI:

Employee / L1 / L2 Support:

- Search and read articles
- Ask the chatbot
- Download attachments

Expert:

- All of the above
- Create new articles (saved as Draft)
- Edit their own articles

Team Lead:

- All of the above
- Approve or reject articles (change status from under_review → published or rejected)
- Monitor article usage

Admin:

- Everything
- Manage users
- Manage permissions
- View analytics

Current Status

Backend — ALREADY BUILT by my teammate ✅

The backend is running on `http://localhost:5000`

Frontend — NOT STARTED ❌

I need to build this from scratch. Node.js is installed on my laptop. I have NOT run `npx create-react-app` yet.

Tech Stack

Layer	Technology
Frontend	React.js 18+
UI Components	Material UI (MUI) v5
Rich Text Editor	React-Quill
HTTP Calls	Axios
Routing	React Router DOM v6
Auth	JWT (token stored in localStorage)
Backend URL	http://localhost:5000

Backend APIs Already Available

Auth

```
POST   http://localhost:5000/api/auth/login
Body: { email, password }
Returns: { token, user: { user_id, email, role, department } }
```

Knowledge Articles

```
GET    http://localhost:5000/api/articles
       → Returns articles filtered by the logged-in user's role automatically

GET    http://localhost:5000/api/articles/:id
       → Get single article + attachments (only if user has permission)

POST   http://localhost:5000/api/articles
       → Create article — Expert, Team Lead, Admin only
       Body: { title, summary, content, module, severity, status, tags,
visibility_role, department }

PUT    http://localhost:5000/api/articles/:id
```

```
       → Update article — Expert (own articles), Team Lead, Admin only

DELETE http://localhost:5000/api/articles/:id
       → Delete article — Admin only

GET    http://localhost:5000/api/articles/search?keyword=MIRO
       → Search articles — results filtered by user's role automatically

POST   http://localhost:5000/api/articles/:id/upload
       → Upload attachment — Expert, Team Lead, Admin only

PATCH  http://localhost:5000/api/articles/:id/status
       → Approve/Reject article — Team Lead, Admin only
       Body: { status: "published" | "rejected" }
```

Article Object Shape

json

```json
{
  "article_id": 1,
  "title": "MIRO Invoice Posting Error",
  "summary": "Short description of the issue",
  "content": "Full rich text content with solution steps",
  "module": "Finance",
  "severity": "high",
  "status": "published",
  "tags": ["MIRO", "invoice", "FI"],
  "visibility_role": "Employee",
  "department": "Finance",
  "created_by": 1,
  "created_at": "2024-01-01T00:00:00Z"
}
```

Attachment Object Shape

json

```
{
  "attachment_id": 1,
  "article_id": 1,
  "file_name": "error_screenshot.png",
  "file_path": "uploads/error_screenshot.png",
  "file_type": "image/png",
  "uploaded_by": 1
}
```

Auth Header (for protected routes)

All POST, PUT, PATCH, DELETE requests need this header:

```
Authorization: Bearer <token>
```

Token is received from login and stored in localStorage as `token`. User object is stored in localStorage as `user` (JSON stringified).

Database Tables (for your reference)

```
users
  → user_id, email, password, role, department

knowledge_articles
  → article_id, title, summary, content, module, severity, status,
    tags, visibility_role, department, created_by

article_attachments
  → attachment_id, article_id, file_name, file_path, file_type, uploaded_by
```

Folder Structure to Create

```
client/
├── public/
│   └── index.html
├── src/
│   ├── api/
│   │   └── articles.js          ← All Axios API call functions
```

```
|   ├── components/
|   |   ├── Sidebar.jsx          ← Left navigation menu
|   |   ├── Topbar.jsx           ← Top header bar with user info + logout
|   |   └── ProtectedRoute.jsx   ← Redirects to /login if not logged in
|   ├── pages/
|   |   ├── Login.jsx            ← Login page
|   |   ├── Dashboard.jsx        ← Home with stats
|   |   ├── ArticleList.jsx      ← List + search all articles (role-
filtered)
|   |   ├── ArticleDetail.jsx    ← View single article + attachments
|   |   ├── ArticleCreate.jsx    ← Create new article (Expert/Admin only)
|   |   └── ArticleEdit.jsx      ← Edit existing article (Expert/Admin
only)
|   ├── context/
|   |   └── AuthContext.jsx      ← Global login/auth state + role info
|   ├── utils/
|   |   └── roleUtils.js         ← Helper functions to check user
permissions
|   ├── App.jsx                  ← Routes
|   └── index.js                 ← Entry point (don't modify)
└── package.json
```

Pages to Build (In This Order)

Order	Page	Route	Who Sees It
1	Login	`/login`	Everyone (public)
2	Sidebar + Layout	(component)	All logged-in users
3	Dashboard	`/`	All logged-in users
4	Article List	`/articles`	All logged-in users (filtered by role)
5	Article Detail	`/articles/:id`	All logged-in users (if permitted)
6	Create Article	`/articles/new`	Expert, Team Lead, Admin only
7	Edit Article	`/articles/:id/edit`	Expert (own), Team Lead, Admin only

RBAC Rules to Implement in the Frontend

Hiding/Showing UI Elements Based on Role

```javascript
// Get current user from localStorage
const user = JSON.parse(localStorage.getItem('user'));
const role = user?.role;

// Helper checks
const isAdmin = role === 'Admin';
const isTeamLead = role === 'Team Lead';
const isExpert = role === 'Expert';
const canCreate = ['Expert', 'Team Lead', 'Admin'].includes(role);
const canApprove = ['Team Lead', 'Admin'].includes(role);
const canDelete = role === 'Admin';
```

What to Show/Hide

UI Element	Who Sees It
"Create Article" button	Expert, Team Lead, Admin
"Edit" button on article	Expert (own articles only), Team Lead, Admin
"Delete" button on article	Admin only
"Approve / Reject" buttons	Team Lead, Admin
Draft articles in list	Expert, Team Lead, Admin
"Under Review" articles	Team Lead, Admin
All articles regardless of dept	Admin only

Article Approval Workflow (Visual)

```
Expert creates article
        ↓
   Status = "draft"
        ↓
```

```
Expert submits for review
        ↓
   Status = "under_review"
        ↓
Team Lead reviews it
        ↓
   Approve → Status = "published"   (visible to all allowed roles)
   Reject  → Status = "rejected"    (back to Expert to fix)
```

In the UI: on the Article Detail page, if the logged-in user is a Team Lead or Admin and the article status is "under_review", show **Approve** and **Reject** buttons.

UI Guidelines — VERY IMPORTANT

This app must look like a **professional enterprise internal tool** — think SAP Fiori, ServiceNow, or Microsoft SharePoint. Clean, corporate, no bright colors, no playful design. Like something a real company would actually use.

Overall Look & Feel

- **Color palette:** Dark navy sidebar (`#0A1929`), white main content area, grey backgrounds (`#F4F6F8`)
- **Primary accent color:** `#1565C0` (dark professional blue) — used for buttons, links, active states
- **Typography:** Use MUI's default font (Roboto). Keep font sizes moderate — nothing too big or flashy
- **No rounded corners everywhere** — use subtle border radius (4px max) to keep it sharp and corporate
- **No gradients, no shadows everywhere** — flat, clean, minimal
- **Spacing:** Generous whitespace, content should not feel cramped

Sidebar

- Dark navy background (`#0A1929`)
- White icons and text
- Active page should be highlighted with a blue left border and slightly lighter background
- Company logo or app name "SAP AMS" at the top in white
- User role shown at the bottom of the sidebar

Topbar

- White background with a thin bottom border (`#E0E0E0`)
- Show app section name (e.g. "Knowledge Base") on the left
- Show logged-in user's name + role chip on the right
- Logout button (icon only) on the far right

Tables (Article List page)

- Clean MUI Table with a light grey header row
- Alternate row shading (very subtle, like `#FAFAFA`)
- No card-style rows — use a proper data table
- Compact row height — enterprise apps show more data per screen
- Status chips: Draft = grey, Under Review = amber/orange, Published = green — all muted shades, not bright

Forms (Create/Edit Article)

- White card with a subtle border, not a floating paper shadow
- Section headings above groups of fields (e.g. "Article Details", "Classification", "Content")
- Fields should be neatly arranged in a 2-column grid where possible
- Save/Submit button in dark blue, Cancel in outlined grey

Dashboard

- Stats shown in clean metric cards — white background, thin border, number in large bold dark text, label below in grey
- No pie charts or flashy graphs — keep it simple for now

General Rules

- Use **Material UI (MUI) v5** for all components
- Use **React-Quill** for the rich text content editor in Create/Edit pages
- Show the logged-in user's **name and role** in the Topbar
- Show **loading spinners** while API calls are in progress
- Show **error messages** in a red MUI Alert box if API calls fail
- If a user tries to access a page they don't have permission for → show a clean "Access Denied" page

- Articles in the list should show a **status chip** (Draft = grey, Under Review = amber, Published = green)

Dropdown Options

SAP Module

```
Finance, HR, Supply Chain, Sales, Procurement, Production, Quality Management, Plant Maintenance
```

Severity

```
critical, high, medium, low
```

Status (only Expert/Admin can set this)

```
draft, under_review, published
```

Visibility Role (who can see this article)

```
Employee, L1 Support, L2 Support, Expert, Team Lead, Admin
```

Department

```
Finance, HR, MM (Materials Management), SD (Sales), PP (Production), QM (Quality), PM (Plant Maintenance)
```

---

npm Packages to Install

After running `npx create-react-app client` and `cd client`, run:

bash

```
npm install @mui/material @emotion/react @emotion/styled @mui/icons-material re
```

How to Send Auth Token with Every Request

javascript

```javascript
const token = localStorage.getItem('token');
const config = {
  headers: { Authorization: `Bearer ${token}` }
};

// GET with auth
axios.get(url, config);

// POST with auth
axios.post(url, data, config);

// PUT with auth
axios.put(url, data, config);

// PATCH with auth
axios.patch(url, data, config);
```

AuthContext — What It Should Store

javascript

```javascript
// After login, store this in context AND localStorage:
{
  token: "jwt_token_here",
  user: {
    user_id: 1,
    email: "user@company.com",
    name: "Ravi Kumar",
    role: "Expert",           // ← very important for RBAC
    department: "Finance"     // ← important for filtering
  }
}
```

Important Notes

- Backend is already running — do NOT touch the backend code

- The backend already filters articles by role — you don't need to filter on the frontend, just hide/show UI buttons based on role
- Build one page/component at a time, test it, then move to the next
- Use `http://localhost:5000` as the base URL for all API calls
- Store JWT token in `localStorage` after login
- Redirect to `/login` if no token is found
- This is for a demo/internship project — clean and functional is more important than perfect

---

How to Start RIGHT NOW

bash

```bash
# Step 1 - Go to your project folder
cd Desktop/sap-ams-app

# Step 2 - Create React app
npx create-react-app client

# Step 3 - Go into it
cd client

# Step 4 - Install all packages
npm install @mui/material @emotion/react @emotion/styled @mui/icons-material re

# Step 5 - Start the app
npm start
```

Then ask the AI to give you code for each file one by one in this order:

1. `src/utils/roleUtils.js`
2. `src/context/AuthContext.jsx`
3. `src/components/ProtectedRoute.jsx`
4. `src/App.jsx`
5. `src/pages/Login.jsx`
6. `src/components/Sidebar.jsx`
7. `src/components/Topbar.jsx`
8. `src/api/articles.js`

9. `src/pages/Dashboard.jsx`
10. `src/pages/ArticleList.jsx`
11. `src/pages/ArticleDetail.jsx`
12. `src/pages/ArticleCreate.jsx`
13. `src/pages/ArticleEdit.jsx`

SAP AMS Knowledge Management Platform | NTT DATA Business Solutions Internship
Stack: React + Node.js + PostgreSQL | Auth: JWT + RBAC